

圏論入門：射・関手・自然変換から Haskell へ

Hiroki Funashima

Saturday, May 2 2026

概要

この資料は、圏論を初めて学ぶ学生に向けて、圏とは何か、射とは何か、関手とは何か、自然変換とは何かを順に説明し、それらがどのようにプログラミング、とくに Haskell に接続するかを学ぶための入門教材である。

圏論は高度で抽象的な数学として語られることが多い。しかし本資料では、圏論を「構造と変換を整理するための言語」として導入する。Haskell を扱う際には、数学的な厳密性を保ちつつも、無理にすべてを圏論で説明しようとはしない。まずは、型、関数、`fmap`、`Maybe`、リストなどを題材として、圏論的な見方がプログラムの構造理解にどのように役立つかを見る。

目次

1	この資料の目標	4
2	なぜ圏論を学ぶのか	4
3	圏とは何か	5
3.1	まずは図で見る	5
3.2	圏の定義	5
3.3	射の合成	5
3.4	恒等射	6
4	基本例：集合の圏 Set	7
5	Haskell と圏：型と関数	8
5.1	型を対象、関数を射と見る	8
5.2	Haskell における合成と恒等関数	8
5.3	Haskell を圏として見るときの注意	8
6	射とは何か	9
6.1	射は「変換」である	9
6.2	射は「構造を保つ写像」である	9
7	関手とは何か	10

7.1	圏から圏への写像	10
7.2	Haskell における関手	10
7.3	Maybe の例	11
7.4	リストの例	11
8	関手法則	12
8.1	Maybe で確かめる	12
8.2	なぜ法則が重要なのか	13
9	自然変換とは何か	14
9.1	関手と関手のあいだの変換	14
9.2	自然性の直観	14
9.3	Haskell における自然変換	14
10	自然変換の例	15
10.1	Maybe からリストへの自然変換	15
10.2	場合分けで確認する	15
10.3	リストから Maybe への自然変換	16
11	プログラミングとの接続	17
11.1	圏論はプログラムの形を説明する	17
11.2	関数合成による設計	17
11.3	文脈を保ったまま関数を適用する	17
11.4	型クラスと法則	18
12	圏論的に見た Haskell の代表例	19
12.1	Maybe : 失敗するかもしれない計算	19
12.2	リスト : 複数の可能性	19
12.3	関数型 : 環境から値を読む	19
13	自然変換と多相性	20
14	圏論をプログラミングに持ち込むと何が嬉しいか	21
14.1	抽象化の水準を上げられる	21
14.2	法則に基づく設計ができる	21
14.3	型から設計意図を読み取れる	21
15	小さな実行例	22
16	演習問題	23
17	参考プログラム	23

17.1	参考プログラム 1：射としての関数と合成	24
17.2	参考プログラム 2：Maybe 関手	24
17.3	参考プログラム 3：自然変換の確認	25
17.4	参考プログラム 4：自作データ型に対する Functor	26
18	追加演習と解答例	27
18.1	演習 1：射と合成	27
18.2	演習 2：恒等射	27
18.3	演習 3：Maybe に対する fmap	28
18.4	演習 4：Functor の合成法則	28
18.5	演習 5：自作データ型の Functor	29
18.6	演習 6：自然変換は一意とは限らない	29
19	まとめ	31

1 この資料の目標

この資料の目標は、圏論を「名前だけ知っている難しい理論」から、「プログラムの構造を説明するための道具」へと近づけることである。

授業の到達目標は次の通りである。

1. 圏とは、対象と射からなる構造であることを説明できる。
2. 射は、単なる関数に限らず、「対象から対象への構造を保つ変換」と見なせることを理解する。
3. 関手は、圏から圏への構造を保つ写像であることを説明できる。
4. Haskell の `Functor` 型クラスと数学的な関手の対応を説明できる。
5. 自然変換を、関手と関手のあいだの「型に依存しない一様な変換」として理解する。
6. `Maybe`、リスト、`fmap`などを用いて、圏論的な考え方がプログラミングにどう現れるかを説明できる。

注意 1.1. 本資料では、Haskell を圏論的に説明するが、Haskell そのものを完全に数学的な圏と同一視するわけではない。実際の Haskell には非停止、未定義値、例外、`seq`、IO などがあるため、数学的な **Set** とは異なる点がある。したがって本資料では、まずは「全域関数だけを考える理想化された Haskell の部分」を念頭に置く。

2 なぜ圏論を学ぶのか

圏論は、もともと数学のさまざまな分野に現れる構造を統一的に扱うために作られた理論である。集合、群、位相空間、ベクトル空間、順序集合などは、一見すると別々の対象に見える。しかしそれらは、いずれも

対象と、対象のあいだの構造を保つ変換

を持っている。

例えば、集合論では対象は集合であり、射は関数である。線形代数では対象はベクトル空間であり、射は線形写像である。位相空間論では対象は位相空間であり、射は連続写像である。

分野	対象	射
集合論	集合	関数
線形代数	ベクトル空間	線形写像
位相空間論	位相空間	連続写像
群論	群	群準同型
プログラミング	型	関数

このように見ると、圏論は「対象そのもの」よりも、「対象のあいだの変換」に注目する理論である。これはプログラミングとも相性がよい。なぜなら、プログラムでは値そのもののだけでなく、値を

どのように変換するかが中心的な問題になるからである。

3 圏とは何か

3.1 まずは図で見る

圏は、対象と射からなる。対象を点、射を矢印として描くと、次のようになる。

$$A \xrightarrow{f} B \xrightarrow{g} C$$

ここで、 A, B, C は対象であり、 f と g は射である。射 f は A から B への射であり、通常

$$f : A \rightarrow B$$

と書く。

3.2 圏の定義

定義 3.1 (圏). 圏 $\mathcal{C}$ は、次のデータからなる構造である。

- (1) 対象の集まり $\text{Ob}(\mathcal{C})$ 。
- (2) 任意の対象 A, B に対して、 A から B への射の集まり $\text{Hom}_{\mathcal{C}}(A, B)$ 。
- (3) 任意の射 $f : A \rightarrow B$ と $g : B \rightarrow C$ に対して、合成射 $g \circ f : A \rightarrow C$ 。
- (4) 各対象 A に対して、恒等射 $\text{id}_A : A \rightarrow A$ 。

さらに、次の2つの法則を満たす。

- (a) 結合法則： $(h \circ g) \circ f = h \circ (g \circ f)$ 。
- (b) 単位法則： $f \circ \text{id}_A = f$ 、かつ、 $\text{id}_B \circ f = f$ 。

この定義だけを見ると抽象的に見えるが、言っていることはそれほど奇妙ではない。

- 射は合成できる。
- 合成の順序は、括弧の付け方に依存しない。
- 何もしない射、つまり恒等射がある。

3.3 射の合成

$f : A \rightarrow B$ と $g : B \rightarrow C$ があるとき、まず f で A から B に行き、次に g で B から C に行くことができる。これを合成して、 A から C への射を作る。

$$A \xrightarrow{f} B \xrightarrow{g} C$$

$\searrow \quad \nearrow$
 $g \circ f$

合成射は

$$g \circ f : A \rightarrow C$$

と書く。注意すべき点は、 $g \circ f$ は「先に f 、次に g 」という意味である。

Haskell では、関数合成は次のように書く。

```
(g . f) x = g (f x)
```

数学の $g \circ f$ と、Haskell の $g . f$ はほぼ同じ意味である。

3.4 恒等射

各対象 A には、何もしない射が存在する。これを恒等射と呼び、 id_A と書く。

$$A \curvearrowright \text{id}_A$$

集合の圏では、恒等射は恒等関数である。

$$\text{id}_A(x) = x$$

Haskell では、恒等関数は `id` である。

```
id :: a -> a  
id x = x
```

4 基本例：集合の圏 **Set**

圏のもっとも基本的な例は、集合の圏 **Set** である。

例 4.1 (集合の圏). **Set** は次のような圏である。

- 対象：集合。
- 射：集合から集合への関数。
- 合成：通常関数合成。
- 恒等射：恒等関数。

例えば、 $A = \mathbb{N}$ 、 $B = \mathbb{N}$ 、 $C = \mathbb{N}$ とし、

$$f(n) = n + 1, \quad g(n) = 2n$$

とする。このとき、 $g \circ f$ は

$$(g \circ f)(n) = g(f(n)) = g(n + 1) = 2(n + 1)$$

である。

これは Haskell では次のように書ける。

```
f :: Int -> Int
f n = n + 1

g :: Int -> Int
g n = 2 * n

h :: Int -> Int
h = g . f
```

この例では、型 `Int` を対象、関数 `f` や `g` を射と見ることができる。

5 Haskell と圏：型と関数

5.1 型を対象、関数を射と見る

Haskell では、`Int`、`Bool`、`String`、`Maybe Int`、`[Char]` などが型である。これらを圏の対象と見なす。
また、Haskell の関数

```
f :: A -> B
```

を、対象 A から対象 B への射と見なす。
例えば、

```
even :: Int -> Bool
```

は、`Int` から `Bool` への射と見ることができる。

$$\text{Int} \xrightarrow{\text{even}} \text{Bool}$$

5.2 Haskell における合成と恒等関数

Haskell では、関数合成 `.` と恒等関数 `id` が標準で用意されている。

```
(.) :: (b -> c) -> (a -> b) -> a -> c  
id  :: a -> a
```

圏論の記法と対応させると、次のようになる。

圏論	Haskell	意味
$f : A \rightarrow B$	<code>f :: A -> B</code>	射・関数
$g \circ f$	<code>g . f</code>	関数合成
id_A	<code>id :: A -> A</code>	恒等射

5.3 Haskell を圏として見るときの注意

注意 5.1 (理想化された Haskell). Haskell の型と関数を圏として見るとき、通常は次のような理想化を行う。

- 非停止する計算をいったん無視する。
- `undefined` や例外をいったん無視する。
- 副作用を持つ計算を純粋な値とは分けて考える。

この理想化のもとでは、Haskell の型を対象、全域関数を射とする圏を考えやすい。この圏をしばしば **Hask** と書く。ただし、実際の Haskell 全体が数学的に素直な圏であると言い切るのは慎重であるべきである。

6 射とは何か

圏論では、射は非常に重要である。圏論は、対象を直接調べるというよりも、対象のあいだにどのような射があるかを調べる理論だと言ってよい。

6.1 射は「変換」である

集合の圏では、射は関数である。

$$f : A \rightarrow B$$

これは、集合 A の要素を集合 B の要素に変換する規則である。

Haskell では、これは次の型に対応する。

```
f :: A -> B
```

例えば、

```
show :: Int -> String
```

は、整数を文字列に変換する射と見ることができる。

6.2 射は「構造を保つ写像」である

重要なのは、射がいつも任意の関数であるとは限らないことである。

例えば、ベクトル空間の圏では、射は線形写像である。任意の関数ではなく、足し算とスカラー倍を保つ関数だけが射になる。

同様に、群の圏では射は群準同型であり、群演算を保つ写像である。

圏	対象	射が保つ構造
Set	集合	特になし
ベクトル空間の圏	ベクトル空間	加法とスカラー倍
群の圏	群	群演算
位相空間の圏	位相空間	位相構造、連続性

この観点から見ると、プログラミングにおける関数も、単に入力を出力に変えるだけではなく、型が持つ構造をどの程度保つかという観点から理解できる。

7 関手とは何か

7.1 圏から圏への写像

関手は、圏から圏への写像である。ただし、対象を対象に写すだけでなく、射も射に写す。

定義 7.1 (関手). 圏 $\mathcal{C}$ から圏 $\mathcal{D}$ への関手 $F : \mathcal{C} \rightarrow \mathcal{D}$ は、次の対応を与える。

- (1) 各対象 $A \in \text{Ob}(\mathcal{C})$ に対して、対象 $FA \in \text{Ob}(\mathcal{D})$ 。
- (2) 各射 $f : A \rightarrow B$ に対して、射 $Ff : FA \rightarrow FB$ 。

さらに、次の2つの法則を満たす。

- (a) 恒等射を保つ : $F(\text{id}_A) = \text{id}_{FA}$ 。
- (b) 合成を保つ : $F(g \circ f) = Fg \circ Ff$ 。

図で表すと、 $\mathcal{C}$ にある射

$$A \xrightarrow{f} B$$

を、関手 F は $\mathcal{D}$ にある射

$$FA \xrightarrow{Ff} FB$$

に写す。

7.2 Haskell における関手

Haskell では、関手に対応する概念が `Functor` 型クラスとして現れる。

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

ここで、`f` は型コンストラクタである。例えば、`Maybe` やリスト `[]` は型コンストラクタである。

- `Maybe` は型 `a` から型 `Maybe a` を作る。
- `[]` は型 `a` から型 `[a]` を作る。

つまり、`Maybe` は対象 `a` を対象 `Maybe a` に写す対応と見ることができる。

さらに、`fmap` は関数

```
g :: a -> b
```

を、関数

```
fmap g :: Maybe a -> Maybe b
```

に持ち上げる。

これを図で書くと次のようになる。

$$a \xrightarrow{g} b$$

$$\text{Maybe } a \xrightarrow[\text{fmap } g]{} \text{Maybe } b$$

上の段は元の関数であり、下の段は `Maybe` の文脈に持ち上げられた関数である。

7.3 `Maybe` の例

`Maybe` は、値が存在するかもしれないし、存在しないかもしれないという状況を表す型である。

```
data Maybe a = Nothing | Just a
```

例えば、文字列を整数に変換する関数が失敗する可能性を持つなら、次のような型になる。

```
readMaybeInt :: String -> Maybe Int
```

`Maybe` に対する `fmap` は、値がある場合だけ中身に関数を適用し、値がない場合は `Nothing` のままにする。

```
fmap (+1) (Just 10) == Just 11
fmap (+1) Nothing  == Nothing
```

7.4 リストの例

リストも `Functor` である。

```
fmap (+1) [1,2,3] == [2,3,4]
fmap length ["aa", "bbb", "c"] == [2,3,1]
```

リストの場合、`fmap` は各要素に関数を適用する。これは通常の `map` と同じである。

```
fmap :: (a -> b) -> [a] -> [b]
fmap = map
```

8 関手法則

Haskell の `Functor` は、単に `fmap` を実装すればよいというだけではない。数学的な関手に対応するためには、次の 2 つの法則を満たす必要がある。

法則 8.1 (Functor の恒等法則). 任意の値 `x` に対して、

$$\text{fmap id } x = \text{id } x$$

が成り立つ。通常は

$$\text{fmap id} = \text{id}$$

と書く。

法則 8.2 (Functor の合成法則). 任意の関数 `f :: a -> b` と `g :: b -> c` に対して、

$$\text{fmap } (g \cdot f) = \text{fmap } g \cdot \text{fmap } f$$

が成り立つ。

この 2 つの法則は、圏論における関手の法則

$$F(\text{id}_A) = \text{id}_{FA}$$

および

$$F(g \circ f) = Fg \circ Ff$$

に対応している。

8.1 `Maybe` で確かめる

`Maybe` について、恒等法則を確認する。

```
fmap id (Just x) = Just (id x) = Just x
fmap id Nothing = Nothing
```

したがって、`fmap id = id` が成り立つ。

次に合成法則を確認する。`Just x` の場合、

```
fmap (g . f) (Just x)
= Just ((g . f) x)
= Just (g (f x))
```

一方で、

```
(fmap g . fmap f) (Just x)
= fmap g (fmap f (Just x))
= fmap g (Just (f x))
= Just (g (f x))
```

となり、一致する。`Nothing` の場合も、どちらも `Nothing` になる。

8.2 なぜ法則が重要なのか

法則は、プログラムの利用者に対して「この型はどのように振る舞う」という約束を与える。

例えば、`Functor` のインスタンスが法則を満たしていれば、次のような変形が安全にできる。

```
fmap g (fmap f xs) == fmap (g . f) xs
```

これは最適化にも、リファクタリングにも、プログラムの意味の説明にも関係する。圏論的な見方は、単に抽象的な飾りではなく、プログラム変換の正当性を支える考え方になる。

9 自然変換とは何か

9.1 関手と関手のあいだの変換

関手は圏から圏への写像であった。では、関手どうしを変換するにはどうすればよいだろうか。

自然変換は、関手と関手のあいだの変換である。

定義 9.1 (自然変換). $F, G: \mathcal{C} \rightarrow \mathcal{D}$ を 2 つの関手とする。 F から G への自然変換 $\alpha: F \Rightarrow G$ とは、各対象 A に対して、 $\mathcal{D}$ の射

$$\alpha_A: FA \rightarrow GA$$

を与える対応であり、任意の射 $f: A \rightarrow B$ について、次の図式が可換になるものをいう。

$$\begin{array}{ccc} FA & \xrightarrow{Ff} & FB \\ \alpha_A \downarrow & & \downarrow \alpha_B \\ GA & \xrightarrow{Gf} & GB \end{array}$$

つまり、

$$Gf \circ \alpha_A = \alpha_B \circ Ff$$

が成り立つ。

この条件を自然性という。

9.2 自然性の直観

自然性の図式は、次の 2 つの経路が同じ結果になることを意味する。

- (1) 先に F の世界で f を適用し、その後で G の世界へ変換する。
- (2) 先に F の世界から G の世界へ変換し、その後で G の世界で f を適用する。

図式でいうと、上に行ってから右に下がる経路と、下に行ってから右に進む経路が一致する。

この条件は、変換 α が対象 A の具体的な中身に依存せず、一様に定義されていることを表している。

9.3 Haskell における自然変換

Haskell では、自然変換はしばしば次のような型を持つ多相関数として現れる。

```
alpha :: forall a. F a -> G a
```

ここで、 F と G はどちらも `Functor` であると考えてる。

重要なのは、`alpha` が任意の型 a に対して同じ形で定義されていることである。 a の具体的な型が `Int` なのか `String` なのかに応じて、特別な処理を勝手に変えることはできない。

この「型に依存しない一様性」が、自然変換の直観に対応する。

10 自然変換の例

10.1 Maybe からリストへの自然変換

Maybe からリストへの変換を考える。

```
maybeToList :: Maybe a -> [a]
maybeToList Nothing = []
maybeToList (Just x) = [x]
```

この関数は、任意の型 `a` に対して定義されている。したがって、型としては

```
maybeToList :: forall a. Maybe a -> [a]
```

と見ることができる。

これは、関手 `Maybe` から関手 `[]` への自然変換と見なせる。

自然性条件は、任意の関数

```
f :: a -> b
```

に対して、次が成り立つことを意味する。

```
map f . maybeToList == maybeToList . fmap f
```

図で書くと、次の可換図式である。

$$\begin{array}{ccc} \text{Maybe } a & \xrightarrow{\text{fmap } f} & \text{Maybe } b \\ \text{maybeToList} \downarrow & & \downarrow \text{maybeToList} \\ [a] & \xrightarrow{\text{map } f} & [b] \end{array}$$

10.2 場合分けで確認する

`Nothing` の場合、左辺は

```
(map f . maybeToList) Nothing
= map f []
= []
```

右辺は

```
(maybeToList . fmap f) Nothing
= maybeToList Nothing
= []
```

であり、一致する。

`Just x` の場合、左辺は

```
(map f . maybeToList) (Just x)
= map f [x]
= [f x]
```

右辺は

```
(maybeToList . fmap f) (Just x)
= maybeToList (Just (f x))
= [f x]
```

であり、やはり一致する。

10.3 リストから `Maybe` への自然変換

逆向きの例として、リストの先頭要素を取り出す関数を考える。

```
listToMaybe :: [a] -> Maybe a
listToMaybe [] = Nothing
listToMaybe (x:_) = Just x
```

これも任意の型 `a` に対して一様に定義されているため、リスト関手から `Maybe` 関手への自然変換と見なせる。

自然性条件は次の式である。

```
fmap f . listToMaybe == listToMaybe . map f
```

図式は次の通りである。

$$\begin{array}{ccc} [a] & \xrightarrow{\text{map } f} & [b] \\ \text{listToMaybe} \downarrow & & \downarrow \text{listToMaybe} \\ \text{Maybe } a & \xrightarrow{\text{fmap } f} & \text{Maybe } b \end{array}$$

11 プログラミングとの接続

11.1 圏論はプログラムの形を説明する

圏論を学ぶと、プログラムの中に現れる次のような構造を統一的に説明できる。

- 関数合成。
- 型コンストラクタ。
- `fmap` による文脈付きの変換。
- `Maybe` やリストのような計算文脈。
- 型に依存しない一様な変換。

特に Haskell では、型がプログラムの意味を強く表すため、圏論的な語彙がプログラム設計の説明に使いやすい。

11.2 関数合成による設計

Haskell では、小さな関数を合成して大きな処理を作ることが多い。

```
normalize :: String -> String
normalize = unwords . words
```

これは、まず `words` によって文字列を単語のリストに分解し、次に `unwords` によって正規化された空白を持つ文字列に戻す関数である。

圏論的には、これは射の合成である。

$$\text{String} \xrightarrow{\text{words}} [\text{String}] \xrightarrow{\text{unwords}} \text{String}$$

11.3 文脈を保ったまま関数を適用する

`Maybe` やリストは、「普通の値」ではなく「文脈付きの値」を表す。

- `Maybe a` は、失敗するかもしれない `a`。
- `[a]` は、複数個の `a`。
- `IO a` は、入出力を伴って得られる `a`。

`fmap` は、普通の関数を文脈付きの値に適用できるようにする。

```
(+1)      :: Int -> Int
fmap (+1) :: Maybe Int -> Maybe Int
fmap (+1) :: [Int] -> [Int]
```

ここで重要なのは、`fmap` が文脈を壊さないことである。`Maybe` なら `Nothing` は `Nothing` のままであり、リストならリストの形を保ったまま各要素を変換する。

11.4 型クラスと法則

Haskell の型クラスは、単に関数名を共有する仕組みではない。良い型クラスは、満たすべき法則を持つ。

例えば `Functor` には、恒等法則と合成法則がある。

```
fmap id      == id
fmap (g . f) == fmap g . fmap f
```

このような法則は、実行時にコンパイラが自動でチェックしてくれるわけではない。しかし、プログラムの意味を考える上では重要である。

圏論は、このような法則を自然に説明する枠組みを与える。

12 圏論的に見た Haskell の代表例

12.1 Maybe：失敗するかもしれない計算

`Maybe` は、計算が失敗する可能性を型で表す。

```
safeDiv :: Int -> Int -> Maybe Int
safeDiv _ 0 = Nothing
safeDiv x y = Just (x `div` y)
```

`Maybe` を使うと、失敗の可能性を戻り値の型に明示できる。これは、例外を暗黙に投げるよりも、関数の意味を型に近づける設計である。

12.2 リスト：複数の可能性

リスト `[a]` は、複数の値を持つ文脈である。

```
choices :: [Int]
choices = [1,2,3]

results :: [Int]
results = fmap (*10) choices
```

これは、各可能性に対して同じ変換を適用している。

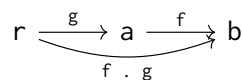
12.3 関数型：環境から値を読む

関数型 `r -> a` も `Functor` である。

```
instance Functor ((->) r) where
  fmap f g = f . g
```

これは、`g :: r -> a` が環境 `r` から値 `a` を読む関数であり、`f :: a -> b` によって結果だけを変換する、という意味である。

図で表すと次のようになる。



13 自然変換と多相性

Haskell で自然変換を理解する上で重要なのが、多相性である。

例えば、次の型を持つ関数を考える。

```
t :: forall a. [a] -> Maybe a
```

この関数は、任意の型 a に対して動かなければならない。そのため、 a の値を勝手に作ることはできない。できることは、入力リストから値を取り出すか、`Nothing` を返すことくらいである。

```
headMaybe :: [a] -> Maybe a
headMaybe []    = Nothing
headMaybe (x:_) = Just x
```

このような関数は、型に対して一様に振る舞う。これが自然変換の直観とよく対応する。

注意 13.1 (パラメトリック性). Haskell の多相関数は、型変数の具体的な中身を知らずに定義される。この性質をパラメトリック性という。自然変換の自然性は、パラメトリック性と深く関係している。ただし、この対応を厳密に扱うには、Haskell の意味論や全域性についてさらに注意が必要である。

14 圏論をプログラミングに持ち込むと何が嬉しいか

14.1 抽象化の水準を上げられる

圏論を使うと、個々のデータ構造ではなく、それらに共通する変換の構造を扱える。

例えば、`Maybe` とリストは別のデータ構造である。しかし、どちらも `Functor` であり、`fmap` によって中身を変換できる。

```
fmap show (Just 10) == Just "10"
fmap show [1,2,3]  == ["1","2","3"]
```

これにより、同じ考え方を異なる文脈に適用できる。

14.2 法則に基づく設計ができる

抽象化には危険もある。抽象化しすぎると、何を意味しているのか分からなくなることがある。

そこで重要になるのが法則である。

`Functor` なら、恒等法則と合成法則がある。これらの法則を満たすことで、`fmap` は「文脈を保った写像」として振る舞うことが保証される。

14.3 型から設計意図を読み取れる

Haskell では、型が設計意図を強く表す。

例えば、

```
parseInt :: String -> Maybe Int
```

という型を見ると、この関数は文字列から整数を作ろうとするが、失敗するかもしれないことが分かる。

また、

```
transform :: Functor f => (a -> b) -> f a -> f b
```

という型を見ると、この関数は具体的な文脈 `f` に依存せず、`Functor` として扱える任意の文脈に対して中身を変換する関数であることが分かる。

15 小さな実行例

以下は、GHCi で試せる小さな例である。

```
-- Maybe
f1 :: Maybe Int
f1 = fmap (+1) (Just 10)

f2 :: Maybe Int
f2 = fmap (+1) Nothing

-- List
f3 :: [Int]
f3 = fmap (*2) [1,2,3]

-- Function composition
normalize :: String -> String
normalize = unwords . words

-- Natural transformation: Maybe to List
maybeToList' :: Maybe a -> [a]
maybeToList' Nothing = []
maybeToList' (Just x) = [x]

-- Natural transformation: List to Maybe
listToMaybe' :: [a] -> Maybe a
listToMaybe' [] = Nothing
listToMaybe' (x:_) = Just x
```

自然性を GHCi で観察するには、例えば次のように確認できる。

```
f :: Int -> String
f = show

left1 :: [String]
left1 = map f (maybeToList' (Just 10))

right1 :: [String]
right1 = maybeToList' (fmap f (Just 10))

-- left1 == right1
```

16 演習問題

演習 16.1 (圏の基本). 集合の圏 **Set** において、対象、射、合成、恒等射がそれぞれ何に対応するか説明せよ。

演習 16.2 (Haskell と圏). Haskell の型 `Bool`、`Int`、`String` を対象と見たとき、次の関数を射として図式で表せ。

```
not    :: Bool -> Bool
show   :: Int  -> String
length :: String -> Int
```

演習 16.3 (関数合成). 次の 2 つの関数を合成して、`String -> Bool` 型の関数を作れ。

```
length :: String -> Int
even    :: Int   -> Bool
```

演習 16.4 (Functor 法則). `Maybe` に対して、`fmap id = id` が成り立つことを、`Nothing` と `Just x` の 2 つの場合に分けて確認せよ。

演習 16.5 (リスト関手). リストに対して、次の式が成り立つことを具体例で確認せよ。

```
fmap (g . f) xs == (fmap g . fmap f) xs
```

演習 16.6 (自然変換). 次の関数が自然変換と見なせる理由を説明せよ。

```
listToMaybe :: [a] -> Maybe a
```

また、自然性条件

```
fmap f . listToMaybe == listToMaybe . map f
```

が何を意味するか説明せよ。

演習 16.7 (型からできることを考える). 次の型を持つ関数として、どのような実装が考えられるか挙げよ。

```
t :: forall a. Maybe a -> [a]
```

また、その実装が型 `a` の具体的な中身に依存できない理由を説明せよ。

17 参考プログラム

この節では、学生が実際に GHCi で試せる小さな参考プログラムを示す。目的は、圏論の概念を Haskell 上で完全に形式化することではなく、射、関手、自然変換という語彙が、プログラムのどの部分に現れるかを確認することである。

17.1 参考プログラム 1：射としての関数と合成

```
module CategoryBasic where

-- f : Int -> Int
f :: Int -> Int
f x = x + 1

-- g : Int -> String
g :: Int -> String
g x = "value = " ++ show x

-- h = g . f : Int -> String
h :: Int -> String
h = g . f

-- identity morphism
sameInt :: Int -> Int
sameInt = id
```

GHCi では次のように確認できる。

```
> h 10
"value = 11"

> sameInt 5
5
```

ここで、 f 、 g 、 h はいずれも射であり、 h は射の合成として得られている。

17.2 参考プログラム 2：Maybe 関手

```
module MaybeFunctorExample where

inc :: Int -> Int
inc x = x + 1

toLabel :: Int -> String
toLabel x = "n = " ++ show x

ex1 :: Maybe Int
ex1 = fmap inc (Just 10)

ex2 :: Maybe Int
ex2 = fmap inc Nothing
```



```
ex3 :: Maybe String
ex3 = fmap (toLabel . inc) (Just 10)

ex4 :: Maybe String
ex4 = (fmap toLabel . fmap inc) (Just 10)
```

この例では、ex3 と ex4 が同じ値になる。これは、関手の合成法則

```
fmap (g . f) == fmap g . fmap f
```

の具体例である。

17.3 参考プログラム 3：自然変換の確認

```
module NaturalTransformationExample where

maybeToList' :: Maybe a -> [a]
maybeToList' Nothing = []
maybeToList' (Just x) = [x]

listToMaybe' :: [a] -> Maybe a
listToMaybe' [] = Nothing
listToMaybe' (x:_) = Just x

f :: Int -> String
f x = "n = " ++ show x

leftMaybeToList :: Maybe Int -> [String]
leftMaybeToList = map f . maybeToList'

rightMaybeToList :: Maybe Int -> [String]
rightMaybeToList = maybeToList' . fmap f

leftListToMaybe :: [Int] -> Maybe String
leftListToMaybe = fmap f . listToMaybe'

rightListToMaybe :: [Int] -> Maybe String
rightListToMaybe = listToMaybe' . map f
```

例えば、次のように確認できる。

```
> leftMaybeToList (Just 10)
["n = 10"]

> rightMaybeToList (Just 10)
["n = 10"]

> leftListToMaybe [1,2,3]
```

```
Just "n = 1"
```

```
> rightListToMaybe [1,2,3]
```

```
Just "n = 1"
```

これは、自然変換の可換図式を Haskell の関数合成として確認している例である。

17.4 参考プログラム 4：自作データ型に対する Functor

```
module BoxFunctorExample where
```

```
newtype Box a = Box a
```

```
    deriving (Eq, Show)
```

```
instance Functor Box where
```

```
    fmap f (Box x) = Box (f x)
```

```
inc :: Int -> Int
```

```
inc x = x + 1
```

```
toLabel :: Int -> String
```

```
toLabel x = "n = " ++ show x
```

```
exBox1 :: Box Int
```

```
exBox1 = fmap inc (Box 10)
```

```
exBox2 :: Box String
```

```
exBox2 = fmap toLabel exBox1
```

```
exBox3 :: Box String
```

```
exBox3 = fmap (toLabel . inc) (Box 10)
```

Box は中身を 1 つだけ持つ単純な文脈である。fmap は、箱そのものを壊さずに、中身だけを変換する。

18 追加演習と解答例

この節では、圏論の定義理解から Haskell による確認までを扱う追加演習を示す。授業では、解答例を後半または別紙に分けてもよい。

18.1 演習 1：射と合成

演習 18.1. 次の Haskell 関数を考える。

```
double :: Int -> Int
double x = 2 * x

toText :: Int -> String
toText x = show x

addBang :: String -> String
addBang s = s ++ "!"
```

- (1) それぞれの関数を、圏論の射として図式で表せ。
- (2) `addBang . toText . double` の型を書け。
- (3) `(addBang . toText . double) 7` の結果を求めよ。

■解答例 射は次のように表せる。

$$\text{Int} \xrightarrow{\text{double}} \text{Int} \xrightarrow{\text{toText}} \text{String} \xrightarrow{\text{addBang}} \text{String}$$

合成の型は次の通りである。

```
addBang . toText . double :: Int -> String
```

計算結果は次の通りである。

```
(addBang . toText . double) 7
= addBang (toText (double 7))
= addBang (toText 14)
= addBang "14"
= "14!"
```

18.2 演習 2：恒等射

演習 18.2. 任意の関数 `f :: a -> b` について、Haskell の `id` が次を満たすことを説明せよ。

```
f . id == f
id . f == f
```

■解答例 任意の値 $x :: a$ に対して、

```
(f . id) x
= f (id x)
= f x
```

である。また、

```
(id . f) x
= id (f x)
= f x
```

である。したがって、`id` は合成に関する単位元として働く。圏論的には、これは恒等射の単位法則である。

18.3 演習 3 : Maybe に対する fmap

演習 18.3. 次の値を計算せよ。

```
fmap (+3) (Just 4)
fmap (+3) Nothing
fmap length (Just "abc")
fmap length Nothing
```

■解答例

```
fmap (+3) (Just 4)      == Just 7
fmap (+3) Nothing      == Nothing
fmap length (Just "abc") == Just 3
fmap length Nothing    == Nothing
```

`Maybe` の `fmap` は、`Just x` の場合だけ中身 x に関数を適用し、`Nothing` の場合は `Nothing` のままにする。

18.4 演習 4 : Functor の合成法則

演習 18.4. 次の関数を考える。

```
f :: Int -> Int
f x = x + 1

g :: Int -> String
g x = show (2 * x)
```

`Maybe` に対して、次の 2 つが同じ結果になることを `Just 10` と `Nothing` の場合に分けて確認せよ。

```
fmap (g . f)
fmap g . fmap f
```

■解答例 `Just 10` の場合、

```
fmap (g . f) (Just 10)
= Just ((g . f) 10)
= Just (g (f 10))
= Just (g 11)
= Just "22"
```

一方、

```
(fmap g . fmap f) (Just 10)
= fmap g (fmap f (Just 10))
= fmap g (Just 11)
= Just (g 11)
= Just "22"
```

であり、一致する。`Nothing` の場合は、どちらも `Nothing` になる。

18.5 演習 5：自作データ型の Functor

演習 18.5. 次のデータ型に対して、`Functor` インスタンスを定義せよ。

```
data Pair a = Pair a a
    deriving (Eq, Show)
```

また、次の値を計算せよ。

```
fmap (+1) (Pair 10 20)
fmap show (Pair 3 5)
```

■解答例 `Pair a` は、同じ型 `a` の値を 2 つ持つデータ型である。したがって、`fmap` は 2 つの中身の両方に関数を適用すればよい。

```
data Pair a = Pair a a
    deriving (Eq, Show)

instance Functor Pair where
    fmap f (Pair x y) = Pair (f x) (f y)
```

計算結果は次の通りである。

```
fmap (+1) (Pair 10 20) == Pair 11 21
fmap show (Pair 3 5)   == Pair "3" "5"
```

18.6 演習 6：自然変換は一意とは限らない

演習 18.6. 次の関数は、`Maybe` からリストへの自然変換と見なせるか。

```
duplicateMaybe :: Maybe a -> [a]
duplicateMaybe Nothing = []
duplicateMaybe (Just x) = [x, x]
```

■解答例 この関数は自然変換と見なせる。任意の型 `a` に対して同じ形で定義されており、型 `a` の具体的な中身には依存していない。

自然性を `Just x` の場合で確認すると、

```
(map f . duplicateMaybe) (Just x)
= map f [x, x]
= [f x, f x]
```

であり、一方で、

```
(duplicateMaybe . fmap f) (Just x)
= duplicateMaybe (Just (f x))
= [f x, f x]
```

である。`Nothing` の場合もどちらも空リストになる。したがって自然性を満たす。

この例から、自然変換は一意とは限らないことが分かる。

19 まとめ

本資料では、圏論の基本概念である圏、射、関手、自然変換を、Haskell との関係を意識しながら説明した。

要点をまとめる。

- 圏は、対象と射からなる構造である。
- 射は合成でき、恒等射を持つ。
- Haskell では、型を対象、関数を射と見ることができる。
- 関手は、対象と射を構造を保って写すものである。
- Haskell の `Functor` は、型コンストラクタと `fmap` によって、関手の考え方をプログラミングに取り入れている。
- 自然変換は、関手と関手のあいだの一様な変換である。
- Haskell の多相関数は、自然変換の直観を理解するうえでよい例になる。

圏論は、最初は抽象的に見える。しかし、Haskell の型、関数、`fmap`、`Maybe`、リストなどを通して見ると、「構造を保つ変換をどう扱うか」という非常に実践的な問題とつながっていることが分かる。

圏論をプログラミングに応用する第一歩は、難しい用語を暗記することではない。むしろ、次の問いを持つことである。

このプログラムは、何を対象とし、どのような射を合成し、どの構造を保っているのか。

この問いを持つことで、プログラムは単なる命令列ではなく、構造を持った設計対象として見えてくる。